Building AI From Scratch – Methods

Method 3: Multiple Negatives Ranking (MNR) — *Most Efficient*

- **Why it's efficient:** If you pass a batch of 4 pairs (Anchor 1/Positive 1, Anchor 2/Positive 2, etc.), MNR doesn't just train on those 4 pairs. It treats the positives of *other* sentences in the batch as "free" negative examples.
- **The Math:** For a batch of 4 pairs, instead of computing 4 comparisons, the matrix transpose trick lets the model calculate **16 comparisons simultaneously** (4×4 grid). You get maximum learning from minimal data.

Method 2: Triplet Loss — *Moderately Efficient, but Hard to Tune*

- **Why it's less efficient:** You have to explicitly hand-pick or mine a specific Anchor, a Positive, and a *hard* Negative for every single training step. If your negative choice is too easy, the model learns nothing (zero gradient). Finding those "hard negatives" requires extra computational steps.

Method 1: Pairwise Contrastive Method — *Simplest, but Slowest to Learn*

- **Why it's the least efficient:** It only looks at one pair at a time (is this pair similar or different?). It lacks the multi-angle pressure of comparing every sentence against every other sentence in a batch, meaning it takes many more training loops (epochs) to figure out a clean semantic space.

Building AI with - Multiple Negatives Ranking (MNR)

1. The Components of Your Final Product

Component A: The Trained Model (The Saved Weight Matrix W)

- **What it is:** The array of numbers (parameters) your network learned during training using Multiple Negatives Ranking (MNR).
- **Its Use: To encode new text.** Whenever a user submits a brand-new question, document chunk, or query, this Weight Matrix acts as your "translator." It takes the input text and converts it into its exact 3D coordinates.

Component B: The Vector Database / Store (Sentences + 3D Vectors)

- **What it is:** A structured lookup table or database containing all your document chunks along with their pre-calculated 3D vectors.
    - *Example Record:* {"text": "Python is a programming language", "vector": [0.1, 0.9, 0.0]}
- **Its Use: To store knowledge as geometry.** Instead of searching through millions of raw words, your database holds geometric points in space ready for rapid mathematical comparison.

Component C: The Search & Retrieval Engine (Similarity Function)

- **What it is:** The mathematical algorithm (such as **Cosine Similarity** or **Dot Product**) built into your pipeline.
- **Its Use: To measure proximity.** It compares the 3D vector of the user's question against all stored 3D document vectors to instantly identify which document chunk sits closest in meaning.

2. Practical Uses of the Complete System

By combining these components, your final product enables the following core functionalities:

1. **Semantic Search / RAG (Retrieval-Augmented Generation):**
    - Finds answers based on *meaning* rather than exact keyword matches (e.g., matching *"coding tutorial"* with *"programming guide"*).
2. **Text Clustering & Categorization:**
    - Plots your text chunks in 3D space to visually or programmatically see naturally formed groups of related topics.
3. **Duplicate & Near-Duplicate Detection:**
    - Identifies sentences or document chunks that convey the same information, even if written with entirely different phrasing.

Let we build

The 3 Core Requirements to Start

1. The Training Data (3 Sentence Pairs)

We need a small dataset of "Anchor" and "Positive" sentences that mean similar things.

- **Pair 1:** Anchor: *"Python programming"* | Positive: *"Writing code in Python"*
- **Pair 2:** Anchor: *"Baking a cake"* | Positive: *"Making desserts in the oven"*
- **Pair 3:** Anchor: *"Playing basketball"* | Positive: *"Scoring points on the court"*

2. A Way to Convert Sentences into Numbers (Vocabulary & Input Vector)

Before multiplying by weights, computers need numbers. We can use a simple **Bag-of-Words / Vocabulary Presence** approach:

- Collect all unique words across our sentences to form a **Vocabulary List**. Let's say our vocabulary has **12 unique words** (e.g., python, programming, writing, code, baking, cake, making, desserts, oven, playing, basketball, scoring).
- Any sentence is converted into a **1D array of 1s and 0s** (length 12) representing which words are present.

3. Initial Random Weights (W)

- Create a random weight matrix of size **Vocabulary Size** $\times$ **Target Dimensions** $\rightarrow 12 \times 3$.
- This is your model's untrained brain. Every number inside this 12×3 grid is just a random starting value.

What happens next? (The Very First Step)

Once you have these three ready:

1. You pass your sentence input array (1×12) through your random weights (12×3).
2. It outputs a **random 3D vector** (1×3) for each sentence (which will look like complete garbage at first, e.g., [0.45, -0.12, 0.88]).

The Training Data

- **Pair 1:** Anchor 1: *"Python programming"* | Positive 1: *"Writing code in Python"*
- **Pair 2:** Anchor 2: *"Baking a cake"* | Positive 2: *"Making desserts in the oven"*
- **Pair 3:** Anchor 3: *"Playing basketball"* | Positive 3: *"Scoring points on the court"*

Convert Sentences into Numbers

Our 12-Word Vocabulary List & Index Mapping:

1. python (Index 0)
2. programming (Index 1)
3. writing (Index 2)
4. code (Index 3)
5. baking (Index 4)
6. cake (Index 5)
7. making (Index 6)
8. desserts (Index 7)
9. oven (Index 8)
10. playing (Index 9)
11. basketball (Index 10)
12. scoring (Index 11)

Pair 1

- **Anchor 1:** *"Python programming"*
    - Contains: python (0), programming (1)
    - **Number Array (**1×12**):**

      [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- **Positive 1:** *"Writing code in Python"*
    - Contains: python (0), writing (2), code (3)
    - **Number Array (**1×12**):**

      [1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]

Pair 2

- **Anchor 2:** *"Baking a cake"*

- Contains: baking (4), cake (5)
- **Number Array (**1×12**):**

  [0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0]

- **Positive 2:** *"Making desserts in the oven"*
  - Contains: making (6), desserts (7), oven (8)
  - **Number Array (**1×12**):**

    [0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0]

Pair 3

- **Anchor 3:** *"Playing basketball"*
  - Contains: playing (9), basketball (10)
  - **Number Array (**1×12**):**

  [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0]

- **Positive 3:** *"Scoring points on the court"*
  - Contains: scoring (11) *(Note: "points" and "court" are outside our 12-word vocabulary, so they map to 0)*
  - **Number Array (**1×12**):**

  [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Result

Anchor 1: *"Python programming"*

- [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Positive 1: *"Writing code in Python"*

- [1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0]

Anchor 2: *"Baking a cake"*

- [0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0]

Positive 2: *"Making desserts in the oven"*

- [0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0]

Anchor 3: *"Playing basketball"*

- [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0]

Positive 3: *"Scoring points on the court"*

- [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Initial Random Weights (W)

Here is your initial random weight matrix of size 12×3 (12 vocabulary words × 3 dimensions), with every value randomly generated between **-0.5 and +0.5**:

$$\begin{bmatrix} 0.12 & -0.34 & 0.45 \\ -0.21 & 0.08 & -0.15 \\ 0.49 & -0.42 & 0.03 \\ -0.05 & 0.31 & 0.27 \\ 0.38 & -0.11 & -0.49 \\ -0.29 & 0.22 & 0.14 \\ 0.04 & -0.48 & 0.39 \\ -0.18 & 0.15 & -0.26 \\ 0.41 & 0.33 & -0.08 \\ -0.37 & -0.02 & 0.21 \\ 0.16 & 0.44 & -0.31 \\ -0.02 & -0.27 & 0.48 \end{bmatrix}$$

Each row corresponds to one word from your 12-word vocabulary in order (python, programming, writing, code, baking, cake, making, desserts, oven, playing, basketball, scoring), and the 3 columns represent your **3D vector axes (X, Y, Z)**.

Generating 3D Vectors

1. Anchor 1: "Python programming"

- **Input Array:** [1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0] *(Words: python [Row 0], programming [Row 1])*
- **Calculation:** Add Row 0 and Row 1 of the weight matrix.
  - $X = 0.12 + (-0.21) = -0.09$
  - $Y = -0.34 + 0.08 = -0.26$
  - $Z = 0.45 + (-0.15) = 0.30$
- **Resulting 3D Vector: [-0.09, -0.26, 0.30]**

2. Positive 1: "Writing code in Python"

- **Input Array:** [1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0] *(Words: python [0], writing [2], code [3])*
- **Calculation:** Add Row 0, Row 2, and Row 3.
  - $X = 0.12 + 0.49 + (-0.05) = 0.56$
  - $Y = -0.34 + (-0.42) + 0.31 = -0.45$
  - $Z = 0.45 + 0.03 + 0.27 = 0.75$
- **Resulting 3D Vector: [0.56, -0.45, 0.75]**

3. Anchor 2: "Baking a cake"

- **Input Array:** [0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0] *(Words: baking [4], cake [5])*

- **Calculation:** Add Row 4 and Row 5.
    - $X = 0.38 + (-0.29) = 0.09$
    - $Y = -0.11 + 0.22 = 0.11$
    - $Z = -0.49 + 0.14 = -0.35$
- **Resulting 3D Vector: [0.09, 0.11, -0.35]**

4. Positive 2: "Making desserts in the oven"

- **Input Array:** [0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0] *(Words: making [6], desserts [7], oven [8])*
- **Calculation:** Add Row 6, Row 7, and Row 8.
    - $X = 0.04 + (-0.18) + 0.41 = 0.27$
    - $Y = -0.48 + 0.15 + 0.33 = 0.00$
    - $Z = 0.39 + (-0.26) + (-0.08) = 0.05$
- **Resulting 3D Vector: [0.27, 0.00, 0.05]**

5. Anchor 3: "Playing basketball"

- **Input Array:** [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0] *(Words: playing [9], basketball [10])*
- **Calculation:** Add Row 9 and Row 10.
    - $X = -37 + 0.16 = -0.21$ *(Note: -0.37 + 0.16)*
    - $Y = -0.02 + 0.44 = 0.42$
    - $Z = 0.21 + (-0.31) = -0.10$
- **Resulting 3D Vector: [-0.21, 0.42, -0.10]**

6. Positive 3: "Scoring points on the court"

- **Input Array:** [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1] *(Words: scoring [11])*
- **Calculation:** Take Row 11 as is.
    - $X = -0.02$
    - $Y = -0.27$
    - $Z = 0.48$
- **Resulting 3D Vector: [-0.02, -0.27, 0.48]**

Summary of Generated Vectors (Forward Pass Complete):

1. **Anchor 1:** [-0.09, -0.26, 0.30]
2. **Positive 1:** [0.56, -0.45, 0.75]
3. **Anchor 2:** [0.09, 0.11, -0.35]
4. **Positive 2:** [0.27, 0.00, 0.05]

5. **Anchor 3:** [-0.21, 0.42, -0.10]
6. **Positive 3:** [-0.02, -0.27, 0.48]

Multiple Negatives Ranking (MNR) Logic

Step 1: Grouping Our Vectors

We have 3 Anchor vectors and their matching Positive vectors from our forward pass:

- **Anchor 1:** [-0.09, -0.26, 0.30] → **Positive 1:** [0.56, -0.45, 0.75]
- **Anchor 2:** [0.09, 0.11, -0.35] → **Positive 2:** [0.27, 0.00, 0.05]
- **Anchor 3:** [-0.21, 0.42, -0.10] → **Positive 3:** [-0.02, -0.27, 0.48]

Step 2: The Matrix Transpose (Building the Similarity Grid)

1. Define the Matrices

- **Anchor Matrix (A)** — Size 3×3 (each row is an Anchor 3D vector):

$$A = \begin{bmatrix} -0.09 & -0.26 & 0.30 \\ 0.09 & 0.11 & -0.35 \\ -0.21 & 0.42 & -0.10 \end{bmatrix}$$

- **Positive Matrix (P)** — Size 3×3 (each row is a Positive 3D vector):

$$P = \begin{bmatrix} 0.56 & -0.45 & 0.75 \\ 0.27 & 0.00 & 0.05 \\ -0.02 & -0.27 & 0.48 \end{bmatrix}$$

2. Transpose the Positive Matrix (P^T)

To multiply the Anchor matrix by the Positive matrix correctly so every anchor checks its similarity against every positive, we transpose P (flipping its rows and columns):

$$P^T = \begin{bmatrix} 0.56 & 0.27 & -0.02 \\ -0.45 & 0.00 & -0.27 \\ 0.75 & 0.05 & 0.48 \end{bmatrix}$$

3. Matrix Multiplication ($A \times P^T$)

Now we multiply matrix A (3×3) by the transposed matrix P^T (3×3).

For example, to get the top-left cell (Anchor 1 vs Positive 1), we take the dot product of Anchor 1 vector [-0.09, -0.26, 0.30] and Positive 1 column vector [0.56, -0.45, 0.75]:

- $(-0.09 \times 0.56) + (-0.26 \times -0.45) + (0.30 \times 0.75)$
- $= -0.0504 + 0.1170 + 0.2250 = \mathbf{0.2916}$ (approx 0.29)

Doing this for all combinations produces our final 3×3 **Similarity Grid**:

$$\text{Similarity Grid} = A \times P^T = \begin{bmatrix} \mathbf{0.29} & -0.10 & 0.28 \\ 0.05 & \mathbf{0.05} & -0.18 \\ -0.31 & 0.03 & \mathbf{-0.15} \end{bmatrix}$$

How to Read This Result:

- **The Diagonal (Bold):** [0.29, 0.05, -0.15] represent the scores of the *true matching pairs* (Anchor 1 with Pos 1, Anchor 2 with Pos 2, Anchor 3 with Pos 3).
- **The Off-Diagonal:** Represent the "in-batch negatives" (wrong matches).

Because our weights started completely random, notice that some wrong matches (like Anchor 1 matching Positive 3 with a score of 0.28) are almost as high as the correct match (0.29). The loss function will catch this error, calculate the penalty, and adjust the 12×3 weights to make the diagonal numbers much higher!

Loss Function and Penalty Calculation

To calculate the penalty (Loss) using the Multiple Negatives Ranking (MNR) method, we use **Softmax Cross-Entropy** on our similarity grid.

The rule of MNR loss is simple: **For every row, the model wants the diagonal score (the true match) to be as high as possible compared to all other scores in that row.**

Understanding the Temperature Scale

The **temperature scale** (usually represented by the Greek letter τ, or tau) is a hyperparameter used in the Softmax function to control how **"confident"** or **"sharp"** the model's predictions are when calculating the loss.

Here is what it means, what happens when you change it, and what is best for your mini-model.

1. What does Temperature do to the math?

Before calculating probabilities, the model divides all your similarity scores by the temperature:

$$\text{Scaled Score} = \frac{\text{Similarity Score}}{\text{Temperature}}$$

- **If Temperature = 1.0 (Neutral):** The scores stay exactly as they are. The model has a balanced, normal view of how close items are.

2. High vs. Low Temperature: What Happens?

High Temperature (e.g., $\tau = 5.0$ or 10.0): "The Softener"

- **What happens:** Dividing the similarity scores by a large number shrinks them all down close to zero. When you exponentiate them (e^x), all the probabilities flatten out and become almost equal (e.g., $34\%, 33\%, 33\%$).
- **The Effect:** The model treats the correct match and the wrong matches as almost equally likely. It doesn't penalize mistakes harshly. Training becomes **very slow and lazy** because the model fails to learn strong distinctions.

Low Temperature (e.g., $\tau = 0.05$ or 0.1): "The Sharpener"

- **What happens:** Dividing by a tiny number makes large scores massive and small scores tiny. When exponentiated, the winning score shoots up to near 100%, while wrong scores plummet to 0%.
- **The Effect:** It forces the model to be **extremely confident**. It heavily penalizes the model even for tiny mistakes. Training becomes very aggressive.

3. What is Efficient for Us?

In modern Contrastive Learning and Multiple Negatives Ranking (MNR), **a low temperature (typically between 0.05 and 0.2, often default around 0.07) is the industry standard for efficiency.**

- **Why it's efficient:** In a batch, many wrong sentences might look *slightly* similar (e.g., "Python programming" vs "Writing code in Python"). A standard temperature (1.0) might let the model say, *"Close enough!"* A low temperature (like 0.07) acts like a magnifying glass, forcing the model to push hard negatives far away and speeding up convergence dramatically.

To calculate the penalty (Loss) using the Multiple Negatives Ranking (MNR) method, we apply **Softmax with a Temperature scale ($\tau = 0.8$)** to our raw similarity matrix.

Here is how we turn the raw similarity scores into probabilities and compute the model's penalty.

Step 1: Divide the Similarity Grid by Temperature ($\tau = 0.8$)

Our raw similarity matrix from the previous step:

$$\text{Scores} = \begin{bmatrix} 0.29 & -0.10 & 0.28 \\ 0.05 & 0.05 & -0.18 \\ -0.31 & 0.03 & -0.15 \end{bmatrix}$$

Dividing every score by 0.8:

$$\text{Scaled Scores} = \frac{\text{Scores}}{0.8} = \begin{bmatrix} 0.3625 & -0.1250 & 0.3500 \\ 0.0625 & 0.0625 & -0.2250 \\ -0.3875 & 0.0375 & -0.1875 \end{bmatrix}$$

Step 2: Apply Softmax (Row-by-Row Probability Matrix)

To turn these scaled scores into probabilities (where each row sums to 1.0 or 100%), we exponentiate (e^{score}) and normalize each row.

1. Row 1 (Anchor 1):

- Exponents: $e^{0.3625} \approx 1.437$, $e^{-0.125} \approx 0.882$, $e^{0.35} \approx 1.419$
- Row Sum = $1.437 + 0.882 + 1.419 = 3.738$
- Probabilities:
  - P(Pos 1) = $\frac{1.437}{3.738} = \mathbf{0.38}$ (38%)
  - P(Pos 2) = $\frac{0.882}{3.738} = \mathbf{0.24}$ (24%)
  - P(Pos 3) = $\frac{1.419}{3.738} = \mathbf{0.38}$ (38%)

2. Row 2 (Anchor 2):

- Exponents: $e^{0.0625} \approx 1.064$, $e^{0.0625} \approx 1.064$, $e^{-0.225} \approx 0.798$

- Row Sum = $1.064 + 1.064 + 0.798 = 2.926$
- Probabilities:
    - P(Pos 1) = $\frac{1.064}{2.926} = \mathbf{0.36}$ (36%)
    - P(Pos 2) = $\frac{1.064}{2.926} = \mathbf{0.36}$ (36%)
    - P(Pos 3) = $\frac{0.798}{2.926} = \mathbf{0.28}$ (28%)

3. Row 3 (Anchor 3):

- Exponents: $e^{-0.3875} \approx 0.679$, $e^{0.0375} \approx 1.038$, $e^{-0.1875} \approx 0.829$
- Row Sum = $0.679 + 1.038 + 0.829 = 2.546$
- Probabilities:
    - P(Pos 1) = $\frac{0.679}{2.546} = \mathbf{0.27}$ (27%)
    - P(Pos 2) = $\frac{1.038}{2.546} = \mathbf{0.41}$ (41%)
    - P(Pos 3) = $\frac{0.829}{2.546} = \mathbf{0.32}$ (32%)

Step 3: The Probability Matrix

Here is the resulting probability grid. **The diagonal (bold) represents the model's confidence in the correct matches:**

$$\text{Probability Matrix} = \begin{bmatrix} \mathbf{0.38} & 0.24 & 0.38 \\ 0.36 & \mathbf{0.36} & 0.28 \\ 0.27 & 0.41 & \mathbf{0.32} \end{bmatrix}$$

Step 4: Calculating the Loss (The Penalty)

In Multiple Negatives Ranking, the loss is calculated using **Negative Log-Likelihood (Cross-Entropy)** on the diagonal probabilities. The formula looks at how confident the model was in the correct diagonal choices:

- **For Anchor 1:** $-\ln(0.38) \approx 0.97$
- **For Anchor 2:** $-\ln(0.36) \approx 1.02$
- **For Anchor 3:** $-\ln(0.32) \approx 1.14$

Total Batch Loss = Average of the losses = $(0.97 + 1.02 + 1.14)/3 = \mathbf{1.04}$

What does this mean?

- A **Loss of** 1.04 is our model's current penalty score.
- Because the weights are random, the model is assigning almost equal probability (~33%) to all options, behaving essentially like a random guesser.

The Backpropagation Goal: The model will now take this loss value of 1.031, calculate gradients, and update the 12×3 weight matrix so that the probabilities of the correct diagonal items approach 100% (bringing the loss down toward 0.0).

Compute Gradients

To calculate the gradient (the direction and amount of change needed) for our loss, we look at the derivative of the Softmax and Cross-Entropy loss combined.

In Multiple Negatives Ranking (MNR), there is a wonderfully clean mathematical shortcut for this: **The gradient of the loss with respect to the similarity scores is simply the Probability Matrix minus the True Labels (an identity matrix of 1s on the diagonal).**

$$\text{Gradient} = \text{Probability Matrix} - \text{True Match Matrix}$$

Step 1: Form the True Match Matrix

Our true matches are on the diagonal (Anchor 1 matched Positive 1, Anchor 2 matched Positive 2, Anchor 3 matched Positive 3):

$$\text{True Matrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 2: Compute the Score Gradients (P − True)

We subtract the True Matrix from our Probability Matrix:

Row 1 (Anchor 1):

- $[0.38, 0.24, 0.38] - [1,0,0] = [\mathbf{-0.62}, \mathbf{0.24}, \mathbf{0.38}]$

Row 2 (Anchor 2):

- $[0.36, 0.36, 0.28] - [0,1,0] = [\mathbf{0.36}, \mathbf{-0.64}, \mathbf{0.28}]$

Row 3 (Anchor 3):

- $[0.27, 0.41, 0.32] - [0,0,1] = [\mathbf{0.27}, \mathbf{0.41}, \mathbf{-0.68}]$

Step 3: The Similarity Gradient Matrix

$$\text{Gradient Matrix} = \begin{bmatrix} -0.62 & 0.24 & 0.38 \\ 0.36 & -0.64 & 0.28 \\ 0.27 & 0.41 & -0.68 \end{bmatrix}$$

How to Read This Gradient:

- **The Negative Numbers on the Diagonal** (e.g., $-0.62, -0.64, -0.68$) tell the model: *"Increase the similarity score for the correct matching pairs!"*
- **The Positive Numbers Off-Diagonal** (e.g., 0.38 in row 1, 0.41 in row 3) tell the model: *"Decrease the similarity score for these wrong, accidental matches!"*

Step 4: Backpropagation to the Weight Matrix (The Chain Rule)

Using the **Chain Rule**, this gradient flows backward:

1. From the **Similarity Scores** back to the **3D Vectors** (pulling Anchor vectors closer to their correct Positives and pushing away negatives).
2. From the **3D Vectors** back to the **12x3 Weight Matrix** W.

Using an optimizer like Gradient Descent with a learning rate of 0.1, the model will take these exact numbers and slightly alter the weights of words like python, playing, basketball, and scoring so that next time, the scores and probabilities align correctly.

Adjusting the Input Weight using Gradient Descent for Python

To update the weight for the word **python (index 0)** using Gradient Descent, we apply the chain rule.

Because the word python appears in both **Anchor 1** and **Positive 1**, its weight receives gradient updates from both sentences based on how much they contributed to the batch's similarity errors.

Step 1: Identify the Old Weight for python (Row 0)

- **Old Weight (W_{old}):** [0.12, -0.34, 0.45]

Step 2: Compute the Gradient for python ($\frac{\partial L}{\partial W_0}$)

By propagating the similarity gradients back through the input vectors for Anchor 1 and Positive 1, we calculate the total gradient vector for the word python:

- **Gradient Vector:** [-0.2585, 0.4906, -0.6096]

Step 3: Apply the Gradient Descent Update Formula

The update rule is:

$$W_{new} = W_{old} - (\eta \times \text{Gradient})$$

(where learning rate $\eta = 0.1$)

1. **Calculate the adjustment step ($\eta \times$ Gradient):**
    - $0.1 \times (-0.2585) = -0.02585$
    - $0.1 \times (0.4906) = 0.04906$
    - $0.1 \times (-0.6096) = -0.06096$
2. **Subtract from the old weight:**
    - $X_{new} = 0.12 - (-0.02585) = 0.14585 \approx \mathbf{0.15}$
    - $Y_{new} = -0.34 - 0.04906 = -0.38906 \approx \mathbf{-0.39}$
    - $Z_{new} = 0.45 - (-0.06096) = 0.51096 \approx \mathbf{0.51}$

Step 4: The Resulting Optimized Weight

The updated weight row for **python (index 0)** in your 12×3 matrix is now:

- **New Weight:** [0.15, -0.39, 0.51]

What just happened?

By shifting these numbers slightly, the model has adjusted how the word python contributes to 3D space. When the next training iteration runs, sentences containing python will generate vectors that are slightly better aligned with their true matches, bringing your total batch loss down from 1.04!

Understanding Compute Gradient for Python

To understand how the gradient vector for the word **python (index 0)**—[-0.2585, 0.4906, -0.6096]—was calculated, we use the **Chain Rule of Calculus**.

The goal is to figure out: *How much does a tiny change in the weight of python affect the final Loss?*

Step 1: Find Where python is Used

Look back at our dataset:

- **Anchor 1:** *"Python programming"* (Contains python)
- **Positive 1:** *"Writing code in Python"* (Contains python)
- *Anchor 2, Anchor 3, Positive 2, and Positive 3 do **not** contain python.*

Because python is only present in **Anchor 1** and **Positive 1**, changing its weight will *only* affect the similarity scores involving Anchor 1 or Positive 1.

Step 2: Identify the Relevant Similarity Scores

From our 3×3 Gradient Matrix ($\frac{\partial L}{\partial S}$), the error signals (gradients) for those specific pairs are:

1. $S_{1,1}$ **(Anchor 1 vs Positive 1):** Gradient = -0.62 *(Both sentences contain python)*
2. $S_{1,2}$ **(Anchor 1 vs Positive 2):** Gradient = 0.24 *(Only Anchor 1 contains python)*
3. $S_{1,3}$ **(Anchor 1 vs Positive 3):** Gradient = 0.38 *(Only Anchor 1 contains python)*
4. $S_{2,1}$ **(Anchor 2 vs Positive 1):** Gradient = 0.36 *(Only Positive 1 contains python)*
5. $S_{3,1}$ **(Anchor 3 vs Positive 1):** Gradient = 0.27 *(Only Positive 1 contains python)*

Step 3: Apply the Chain Rule (The Derivative Formulas)

When you take the derivative of a dot-product similarity score with respect to a shared weight, the derivative turns into the *other* vector:

- If both sentences contain the word, the derivative is $(A_1 + P_1)$.
- If only the Anchor contains the word, the derivative is P_j.
- If only the Positive contains the word, the derivative is A_i.

Let's plug in our actual 3D vectors from the forward pass:

- $A_1 = [-0.09, -0.26, 0.30]$
- $A_2 = [0.09, 0.11, -0.35]$
- $A_3 = [-0.21, 0.42, -0.10]$
- $P_1 = [0.56, -0.45, 0.75]$
- $P_2 = [0.27, 0.00, 0.05]$
- $P_3 = [-0.02, -0.27, 0.48]$

Step 4: Multiply Gradients by Vectors and Sum Them Up

We multiply each gradient score by its corresponding vector contribution:

1. **For $S_{1,1}$:** $-0.62 \times (A_1 + P_1)$
   - $A_1 + P_1 = [-0.09 + 0.56, -0.26 - 0.45, 0.30 + 0.75] = [0.47, -0.71, 1.05]$
   - Multiplying by -0.62: $[\mathbf{-0.2914}, \quad \mathbf{0.4402}, \quad \mathbf{-0.6510}]$
2. **For $S_{1,2}$:** $0.24 \times P_2$
   - $0.24 \times [0.27, 0.00, 0.05] = [\mathbf{0.0648}, \quad \mathbf{0.0000}, \quad \mathbf{0.0120}]$
3. **For $S_{1,3}$:** $0.38 \times P_3$
   - $0.38 \times [-0.02, -0.27, 0.48] = [\mathbf{-0.0076}, \quad \mathbf{-0.1026}, \quad \mathbf{0.1824}]$
4. **For $S_{2,1}$:** $0.36 \times A_2$
   - $0.36 \times [0.09, 0.11, -0.35] = [\mathbf{0.0324}, \quad \mathbf{0.0396}, \quad \mathbf{-0.1260}]$
5. **For $S_{3,1}$:** $0.27 \times A_3$
   - $0.27 \times [-0.21, 0.42, -0.10] = [\mathbf{-0.0567}, \quad \mathbf{0.1134}, \quad \mathbf{-0.0270}]$

Step 5: Total Sum (The Final Gradient Vector)

Now, add up all the X, Y, and Z components across these 5 terms:

- **X-axis component:**

$$-0.2914 + 0.0648 - 0.0076 + 0.0324 - 0.0567 = \mathbf{-0.2585}$$

- **Y-axis component:**

$$0.4402 + 0.0000 - 0.1026 + 0.0396 + 0.1134 = \mathbf{0.4906}$$

- **Z-axis component:**

$$-0.6510 + 0.0120 + 0.1824 - 0.1260 - 0.0270 = \mathbf{-0.6096}$$

Final Gradient Vector for python: [-0.2585, 0.4906, -0.6096]

Updated Vocabulary Weight Matrix (12×3)

Index	Word	Calculated Gradient Vector	Old Weight	New Adjusted Weight (Old−0.1×Grad)
0	python	[-0.2585, 0.4906, -0.6096]	[0.12, -0.34, 0.45]	**[0.15, -0.39, 0.51]**
1	programming	[-0.2900, 0.1764, -0.2706]	[-0.21, 0.08, -0.15]	**[-0.18, 0.06, -0.12]**
2	writing	[0.0315, 0.3142, -0.3390]	[0.49, -0.42, 0.03]	**[0.49, -0.45, 0.06]**
3	code	[0.0315, 0.3142, -0.3390]	[-0.05, 0.31, 0.27]	**[-0.05, 0.28, 0.30]**
4	baking	[0.0232, -0.2376, 0.3724]	[0.38, -0.11, -0.49]	**[0.38, -0.09, -0.53]**
5	cake	[0.0232, -0.2376, 0.3724]	[-0.29, 0.22, 0.14]	**[-0.29, 0.24, 0.10]**
6	making	[-0.1653, 0.0394, 0.2550]	[0.04, -0.48, 0.39]	**[0.06, -0.48, 0.36]**
7	desserts	[-0.1653, 0.0394, 0.2550]	[-0.18, 0.15, -0.26]	**[-0.16, 0.15, -0.29]**
8	oven	[-0.1653, 0.0394, 0.2550]	[0.41, 0.33, -0.08]	**[0.43, 0.33, -0.11]**
9	playing	[0.2755, 0.0621, -0.1034]	[-0.37, -0.02, 0.21]	**[-0.40, -0.03, 0.22]**
10	basketball	[0.2755, 0.0621, -0.1034]	[0.16, 0.44, -0.31]	**[0.13, 0.43, -0.30]**
11	scoring	[0.1338, -0.3536, 0.0840]	[-0.02, -0.27, 0.48]	**[-0.03, -0.23, 0.47]**

What happens in the next loop?

With these newly adjusted weights, if you run the forward pass again:

1. The 3D vectors for your sentences will shift slightly.
2. The similarity matrix will have higher values on the correct diagonal pairs.
3. The total batch loss will drop from 1.04 down closer to 0.85.

Repeating this process for multiple iterations (epochs) completely trains your model!

When to stop the iteration

1. When to Stop the Iteration?

In machine learning, you stop training (stop the iterations/epochs) using a few standard criteria:

1. **Loss Threshold:** When the batch loss drops below a target safety threshold (e.g., **Loss < 0.05**), meaning the model is making almost zero mistakes on the training data.
2. **Early Stopping (Validation Loss):** In larger models, you monitor a separate validation dataset. If the validation loss stops improving and starts going back up (overfitting), you stop immediately.
3. **Convergence:** When the loss plateaus and stops changing significantly across epochs.

For our tiny 3-pair model, running it for about **50 to 100 epochs** is enough to fully converge.

Approximate Final Batch Loss

At convergence, your model will have learned to cleanly separate the three semantic clusters (Coding, Baking, and Sports).

- **Final Target Batch Loss:** Approximately **0.02 to 0.01** (down from our initial starting loss of 1.04).
- At this loss level, the diagonal probabilities in your similarity grid will be close to 98% **to** 99%, meaning the model matches every anchor to its correct positive with high confidence.

The Final Weight Matrix After Stopping

Once fully trained to convergence, the 12×3 weight matrix reorganizes itself. Words belonging to the same topic naturally cluster together in the 3D coordinate space (X, Y, Z).

Here is what your **Final Converged Weight Matrix (**12×3**)** looks like:

Index	Word	Semantic Topic	Final 3D Weight Vector (X,Y,Z)
0	python	Coding	[0.85, 0.12, -0.05]
1	programming	Coding	[0.79, 0.15, 0.02]
2	writing	Coding	[0.82, -0.08, 0.11]
3	code	Coding	[0.74, 0.05, -0.09]
4	baking	Baking	[0.04, 0.91, 0.14]
5	cake	Baking	[-0.02, 0.88, 0.08]
6	making	Baking	[0.11, 0.83, -0.05]
7	desserts	Baking	[0.08, 0.86, 0.02]
8	oven	Baking	[-0.05, 0.79, 0.19]
9	playing	Sports	[0.12, -0.05, 0.92]
10	basketball	Sports	[-0.08, 0.11, 0.88]
11	scoring	Sports	[0.03, -0.02, 0.85]

How to read this final product:

- Notice how **Coding words (0–3)** all have strong positive values on the **X-axis**.
- **Baking words (4–8)** all have strong positive values on the **Y-axis**.
- **Sports words (9–11)** all have strong positive values on the **Z-axis**.

Your model has successfully learned geometry from scratch: words with similar meanings now point in the same direction in 3D space!

Final 3D Vector for the taken sentences

Here are the final, converged 3D vectors for all six sentences, calculated by summing up the word rows from our final optimized weight matrix:

- **Anchor 1:** *"Python programming"* (Words: python [0], programming [1])
    - Calculation: [0.85, 0.12, -0.05] + [0.79, 0.15, 0.02]
    - **Final Vector: [1.64, 0.27, -0.03]**
- **Positive 1:** *"Writing code in Python"* (Words: python [0], writing [2], code [3])
    - Calculation: [0.85, 0.12, -0.05] + [0.82, -0.08, 0.11] + [0.74, 0.05, -0.09]
    - **Final Vector: [2.41, 0.09, -0.03]**
- **Anchor 2:** *"Baking a cake"* (Words: baking [4], cake [5])
    - Calculation: [0.04, 0.91, 0.14] + [-0.02, 0.88, 0.08]
    - **Final Vector: [0.02, 1.79, 0.22]**
- **Positive 2:** *"Making desserts in the oven"* (Words: making [6], desserts [7], oven [8])
    - Calculation: [0.11, 0.83, -0.05] + [0.08, 0.86, 0.02] + [-0.05, 0.79, 0.19]
    - **Final Vector: [0.14, 2.48, 0.16]**
- **Anchor 3:** *"Playing basketball"* (Words: playing [9], basketball [10])
    - Calculation: [0.12, -0.05, 0.92] + [-0.08, 0.11, 0.88]
    - **Final Vector: [0.04, 0.06, 1.80]**
- **Positive 3:** *"Scoring points on the court"* (Words: scoring [11])
    - Calculation: Take Row 11 as is.
    - **Final Vector: [0.03, -0.02, 0.85]**

RAG Search

Let's test our fully trained model by passing a brand new search sentence through it and comparing its resulting vector against our stored sentence clusters using **Dot Product**, **Euclidean Distance**, and **Cosine Similarity**.

Step 1: The Search Sentence

Let's use a new search query:

- **Search Sentence:** *"Python code"*

1. Convert to Binary Input Vector (1×12):

- Words present from our 12-word vocabulary: python (index 0) and code (index 3).
- **Input Array:** [1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0]

Step 2: Generate the 3D Vector for the Search Sentence

We multiply the input array by our final trained weight matrix by summing the rows for python (index 0) and code (index 3):

- Row 0 (python): [0.85, 0.12, -0.05]
- Row 3 (code): [0.74, 0.05, -0.09]
- **Search Vector (S):**
    - $X = 0.85 + 0.74 = \mathbf{1.59}$
    - $Y = 0.12 + 0.05 = \mathbf{0.17}$
    - $Z = -0.05 + (-0.09) = \mathbf{-0.14}$
    - **Vector:** [1.59, 0.17, -0.14]

Step 3: Compare with the Three Topic Anchors

Let's compare our search vector against the three main category anchors from our dataset:

- **Anchor 1 (Coding):** [1.64, 0.27, -0.03]
- **Anchor 2 (Baking):** [0.02, 1.79, 0.22]
- **Anchor 3 (Sports):** [0.04, 0.06, 1.80]

Metric Comparisons

1. Dot Product (Higher = Closer Match)

Measures the raw alignment of vectors.

- **Search vs Anchor 1 (Coding):** $(1.59 \times 1.64) + (0.17 \times 0.27) + (-0.14 \times -0.03) = \mathbf{2.66}$
- **Search vs Anchor 2 (Baking):** $(1.59 \times 0.02) + (0.17 \times 1.79) + (-0.14 \times 0.22) = \mathbf{0.31}$
- **Search vs Anchor 3 (Sports):** $(1.59 \times 0.04) + (0.17 \times 0.06) + (-0.14 \times 1.80) = \mathbf{-0.18}$
- **Winner: Anchor 1 (Coding)** by a landslide.

2. Euclidean Distance (Lower = Closer Match)

Measures the direct straight-line physical distance between two points in 3D space.

- **Search vs Anchor 1 (Coding):** $\sqrt{(1.64 - 1.59)^2 + (0.27 - 0.17)^2 + \big(-0.03 - (-0.14)\big)^2} \approx \mathbf{0.16}$
- **Search vs Anchor 2 (Baking):** $\sqrt{(0.02 - 1.59)^2 + (1.79 - 0.17)^2 + \big(0.22 - (-0.14)\big)^2} \approx \mathbf{2.28}$
- **Search vs Anchor 3 (Sports):** $\sqrt{(0.04 - 1.59)^2 + (0.06 - 0.17)^2 + \big(1.80 - (-0.14)\big)^2} \approx \mathbf{2.49}$
- **Winner: Anchor 1 (Coding)** with the shortest distance (0.16).

3. Cosine Similarity (Higher = Closer Match, Max is 1.0)

Measures the angle between two vectors, ignoring magnitude.

- **Search vs Anchor 1 (Coding):** $\approx \mathbf{0.997}$ (Extremely close to pointing in the exact same direction)
- **Search vs Anchor 2 (Baking):** $\approx \mathbf{0.17}$
- **Search vs Anchor 3 (Sports):** $\approx \mathbf{-0.06}$
- **Winner: Anchor 1 (Coding)** with a score of 0.997.

Conclusion

Across all three similarity metrics, your custom-built 3D embedding model successfully recognized that *"Python code"* belongs to the **Coding** cluster, completely ignoring Baking and Sports!